

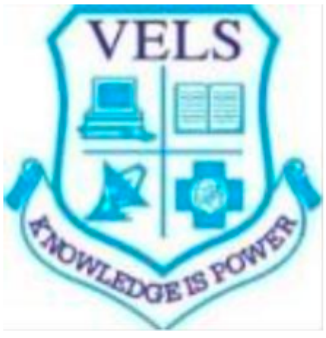
Vision Guard: YOLOv11-Enabled Autonomous Threat Intelligence for Modern Security Infrastructure

FINAL YEAR

PROJECT PHASE-I REPORT

Submitted by
Arnob Samanta (23619103)
M. Akshath (23619101)
A.M Pon Madhu (23619128)
in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY CSE - ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SCHOOL OF ENGINEERING
VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)
CHENNAI
OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project "Vision Guard: YOLOv11-Enabled Autonomous Threat Intelligence for Modern Security Infrastructure" is the bonafide work of **Arnob Samanta (23619103), M. Akshath (23619101), A.M Pon Madhu (23619128**) who carried out the project work under my supervision. Certified further that, to the best of my knowledge, the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD
Dr. K. Ulaga Priya, M.E., Ph.D.
Associate Professor & Head
Department of AI & ML
School of Engineering
VISTAS
Chennai - 600 117

SIGNATURE OF THE SUPERVISOR
Ms. P.Mohana Priya, M.E,(Ph.D)
Assistant Professor
Department of AI & ML
School of Engineering
VISTAS
Chennai - 600 117

Submitted for the Project Work and Viva-Voce examination held on …… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support I received from the positive minds around me in making this venture a successful one. The infrastructural support with all kinds of lab facilities have been a motivating factor in this completion of project work, all because of FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH., with great pleasure that I take this opportunity to thank him. I am grateful to our HEAD OF THE DEPARTMENT, Dr. K. Ulagapriya., for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. I extend our warmest thanks to my INTERNAL GUIDE, Ms. P.Mohana Priya, who has patiently guided and provided us with invaluable advice and help. I am thankful to my Project Coordinator Ms. S.J Kavitha. I would like to express my greatest gratitude to our Staff members, Librarian, and Non-teaching staff members of Computer Centre, who have helped and supported me throughout. Last but not the Least, I wish to thank my parents and my friends for their individual support and interest who has inspired me and encouraged me to go in my own way, without them I would be unable to complete this project.

AKSHATH M (23619101)

ARNOB SAMANTA (23619103)

PON MADHU A.M (23619128)

TABLE OF CONTENTS

	ABSTRACT	6
	LIST OF FIGURES	7
	LIST OF TABLES	8
	LIST OF ABBREVIATIONS	9
Chapter 1	**INTRODUCTION**	10
1.1	Overview of the Project	10
1.2	Background of the Project	10
1.3	Problem Statement	11
1.4	Objectives of the Project	11
1.5	Scope of the Project	12
1.6	Significance of the Project	12
1.7	Organization of the Report	13
Chapter 2	**LITERATURE REVIEW**	14
2.1	Introduction	14
2.2	Traditional Video Surveillance Systems	14
2.3	Deep Learning for Object Detection	14
2.4	YOLOv11 for Threat-Related Object Detection	15
2.5	MOG2 Background Subtraction	15
2.6	CLAHE for Image Enhancement	16
2.7	Temporal Verification	16
2.8	Research Gap	17
Chapter 3	**SYSTEM DESIGN**	18
3.1	Existing System	18
3.2	Proposed System	18
3.3	System Architecture	19
3.4	Hardware Requirements	21
3.5	Software Requirements	21
Chapter 4	**IMPLEMENTATION**	22
4.1	Methodology	22
4.2	Implementation of Major Modules	22
4.2.1	Video Stream Ingestion Module	22
4.2.2	Data Preprocessing and Motion Detection Module	23
4.2.3	CLAHE Image Enhancement Module	23

4.2.4 YOLOv11 Threat Identification Module 23
4.2.5 Temporal Verification Module 24
4.2.6 Event Extraction and Alert Module 24
4.2.7 Dashboard and Monitoring Module 24
4.3 Hardware Setup and System Integration 25
4.4 Integrated System Operation 25
4.5 Experimental Result Verification 26
4.6 Actual Implementation Screenshots 28
4.7 Performance Evaluation and Analysis 30
4.8 Implementation Status and Feature Verification 33
Chapter 5 CONCLUSION AND FUTURE WORK 34
5.1 Conclusion 34
5.2 Future Work 34
5.3 Expected Output 36
5.4 Limitation of the Current System 36
Chapter 6 REFERENCES 37
Chapter 7 APPENDIX 39
7.1 Dataset and Evidence-Record Structure 39
7.2 Installation and Execution Commands 41
7.3 Configuration Examples 42
7.4 Project Architecture 44
Chapter 8 SOURCE CODE 45
8.1 Overview of the Implementation Codebase 45
8.2 Source Code with File Names 45
Chapter 9 CONFERENCE APPLICATION 49

ABSTRACT

Modern video surveillance systems are widely deployed in public places, transportation infrastructure, commercial establishments, and other security-sensitive environments; however, conventional CCTV systems primarily rely on human operators to monitor multiple video streams simultaneously, which can delay threat identification and make continuous surveillance challenging. Vision Guard is proposed as an AI-enabled video surveillance framework that integrates real-time video processing, deep-learning-based object detection, and temporal verification to identify potential weapon-related threats. The framework uses a fine-tuned YOLOv11 object detection model to detect target objects, including firearms and knives, in surveillance video frames. A motion-gating mechanism based on the MOG2 background subtraction algorithm identifies moving foreground regions to reduce unnecessary neural-network inference on static scenes, while Contrast Limited Adaptive Histogram Equalization (CLAHE) can be applied to low-contrast frames to enhance relevant visual features. The detection pipeline is implemented using PyTorch, with Metal Performance Shaders (MPS) acceleration where supported by Apple Silicon hardware. To minimize false alarms caused by isolated detections, camera disturbances, and temporary visual noise, Vision Guard incorporates a temporal verification mechanism that evaluates detections across consecutive frames. When a candidate detection satisfies predefined verification criteria, the system generates structured event information, including the timestamp, camera identifier, detected class, confidence score, and bounding-box coordinates, which can be presented through a monitoring interface for review by authorized security personnel. The conference paper reports improvements in precision from 88.4% to 93.6%, recall from 85.1% to 91.2%, mAP@0.5 from 89.7% to 94.8%, and mAP@0.5:0.95 from 64.2% to 71.5% when comparing baseline YOLOv11 with the proposed fine-tuned pipeline incorporating CLAHE. The reported average processing latency also decreases from 22 milliseconds per frame to 14 milliseconds per frame under the evaluated conditions; however, these figures must be verified against the underlying experimental logs before being considered independently established results. Overall, Vision Guard demonstrates the potential of combining motion-aware preprocessing, deep-learning-based object detection, and temporal verification into a practical surveillance decision-support workflow designed to assist security personnel rather than replace human judgment. Further evaluation across diverse cameras, lighting conditions, environments, and deployment platforms is necessary to establish the system's reliability and generalizability.

Keywords: Artificial Intelligence, Computer Vision, YOLOv11, Weapon Detection, CCTV Surveillance, MOG2 Background Subtraction, CLAHE, Temporal Verification, Deep Learning.

LIST OF FIGURES

Figure	Figure Name	Page NO
3.1	**System Architecture**	**19**
4.1	**Performance Comparison of Baseline YOLOv11 and Vision Guard Pipeline**	**27**
4.2	**Video input**	**30**
4.3	**Weapon Detection**	**31**
4.4	**Output image**	**32**
7.1	**Project Architecture & File Hierarchy**	**44**

LIST OF TABLES

Table No.	Title	Page NO
3.1	**Hardware Components**	**21**
3.2	**Software Components**	**21**
4.1	**Event Extraction Description**	**24**
4.2	**Functional Testing**	**25**
4.3	**Performance Comparison**	**26**
4.4	**Evaluation Record**	**29**
4.5	**Screenshot Evidence Table**	**32**
4.6	**Status and Feature verification**	**33**
7.1	**Dataset Components and Description**	**39**
7.2	**Evidence Field and Description**	**40**

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
CCTV	Closed-Circuit Television
CLAHE	Contrast Limited Adaptive Histogram Equalization
CV	Computer Vision
FPS	Frames Per Second
IoU	Intersection over Union
mAP	Mean Average Precision
MOG2	Mixture of Gaussians 2
NMS	Non-Maximum Suppression
OpenCV	Open Source Computer Vision Library
YOLO	You Only Look Once

CHAPTER 1: INTRODUCTION

1.1 Overview of the Project

Video surveillance has become an important component of modern security infrastructure. CCTV cameras are commonly installed in educational institutions, shopping centres, railway stations, airports, offices, public buildings and other locations that require continuous observation.Although cameras can record activities continuously, traditional surveillance systems generally depend on security personnel to observe video feeds and identify suspicious events. When multiple cameras operate simultaneously, manually monitoring every stream becomes difficult. Important visual events may be overlooked, and reviewing recorded footage can require considerable time.

Artificial intelligence and computer vision provide opportunities to automate parts of this process. Object detection models can analyse video frames and identify specified objects, while image-processing techniques can improve the input quality and reduce unnecessary computations.Vision Guard is designed as an intelligent video surveillance framework that combines these techniques into a unified processing pipeline.

The system uses YOLOv11 for target-object detection. Before deep-learning inference, the MOG2 background subtraction algorithm identifies moving regions within the video. Frames that satisfy the configured motion criteria can proceed to further processing, while static scenes can bypass expensive neural-network inference.CLAHE is used to improve local contrast in frames that require visibility enhancement. The resulting frames are passed to the detection model, which predicts the classes and locations of relevant objects.

The system then evaluates detections across consecutive frames to reduce the likelihood of generating alerts from isolated false positives. Verified events can be recorded and displayed through a monitoring interface.The principal objective is to connect video acquisition, image preprocessing, object detection, temporal verification and event reporting into one surveillance workflow.Vision Guard is a decision-support system. Its detections indicate potential objects of interest and should be reviewed by an authorized human operator before consequential action is taken.

1.2 Background of the Project

Deep learning has significantly influenced computer vision by enabling models to learn visual features directly from training images.Object detection differs from image classification because it identifies both the class of an object and its location within an image. These locations are generally represented using bounding boxes.In a surveillance environment, an object detection system can process video frames and identify target objects that may require further attention.However, deploying object detection in continuous video surveillance introduces several challenges.

First, changing illumination: Cameras may operate during daylight, at night, or under artificial lighting. Poor visibility can make small objects difficult to detect.

Second, computational overhead: Processing every frame through a deep neural network can consume substantial computing resources, particularly when multiple high-resolution streams are involved.

Third, transient detections: Camera vibration, motion blur, occlusion and background disturbances can produce incorrect detections in individual frames.

Fourth, event management: A detection is more useful when its timestamp, camera identity, confidence score and location are recorded consistently.

Vision Guard addresses these challenges through motion-aware preprocessing, optional contrast enhancement, deep-learning inference and temporal verification.

1.3 Problem Statement

Traditional CCTV systems rely heavily on continuous human observation, while basic automated surveillance systems may process every frame without considering whether meaningful movement has occurred.This creates challenges in computational efficiency, detection reliability and timely event identification.

The major problems addressed by Vision Guard are:

1. Continuous manual monitoring of multiple CCTV streams is demanding.
2. Low-light and low-contrast conditions can reduce object-detection performance.
3. Processing every video frame through a neural network can increase computational overhead.
4. Isolated false-positive detections can produce unnecessary alerts.
5. Detection results require structured event information to support efficient review.
6. Surveillance operators need a clear interface for examining detected events and their supporting visual evidence.

Problem statement: To design and evaluate an AI-enabled video surveillance framework that combines motion-gated preprocessing, YOLOv11-based object detection and temporal verification to identify potential weapon-related threats while considering computational efficiency and false-positive reduction.

1.4 Objectives of the Project

The main objective is to develop an intelligent surveillance prototype capable of detecting predefined target objects in video streams and presenting the results for human review.

The specific objectives are:

1. To acquire video from a supported camera or prerecorded video file.
2. To use MOG2 background subtraction to identify moving foreground regions.
3. To reduce unnecessary model inference on frames classified as static.
4. To apply CLAHE when contrast enhancement is required.
5. To use a fine-tuned YOLOv11 model for detecting supported target classes.
6. To generate bounding boxes, class labels and confidence scores.
7. To verify candidate detections across consecutive frames.
8. To record event metadata, including timestamps and camera identifiers where available.
9. To provide a monitoring interface for displaying detection results and event history.
10. To evaluate detection performance and processing latency using an appropriate test dataset.

These objectives should be assessed against the functions actually implemented in your project.

1.5 Scope of the Project

The project focuses on intelligent video surveillance and the automated identification of predefined visual targets.

Its scope includes:

- Video acquisition from supported sources.
- Frame extraction and preprocessing.
- Motion detection using MOG2.
- Conditional image enhancement using CLAHE.
- Object detection using YOLOv11.
- Temporal verification of candidate detections.
- Event metadata generation.
- Display of detection results and associated video frames.
- Performance evaluation using object-detection metrics.

The project can be extended to support multiple cameras, centralized event storage and remote monitoring.

However, these extensions should be described as implemented features only if they have actually been developed and tested.The current project description does not establish that Vision Guard performs face recognition, determines criminal intent, identifies individuals or autonomously dispatches emergency services. Those capabilities should not be claimed in the report.

1.6 Significance of the Project

Vision Guard demonstrates the integration of computer vision and deep learning into an intelligent surveillance workflow.

Its significance can be explained through four major aspects.

Automated detection: The system can analyse video frames to identify predefined objects and highlight their locations.

Computational efficiency: Motion gating can avoid unnecessary deep-learning inference when a scene is classified as static. The actual reduction in computation depends on the camera environment and motion-gating configuration.

Temporal verification: Evaluating detections across multiple frames can help suppress isolated false positives.

Structured monitoring: Recording detection information in a consistent format can help authorized personnel review events and investigate the associated footage.

The project also provides a foundation for studying real-time object detection, computer vision preprocessing, model evaluation and software integration.

1.7 Organization of the Report

The report is divided into five chapters.

- **Chapter 1 – Introduction:** Presents the background, problem statement, objectives, scope and significance of Vision Guard.
- **Chapter 2 – Literature Review:** Reviews object detection, deep-learning-based surveillance, image enhancement and temporal verification techniques.
- **Chapter 3 – System Design:** Explains the proposed architecture, functional modules, hardware requirements and software requirements.
- **Chapter 4 – Implementation:** Describes the processing pipeline, integration of the modules, testing methodology and experimental results.
- **Chapter 5 – Conclusion and Future Work:** Summarizes the project, discusses its limitations and identifies possible improvements.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

The development of intelligent surveillance systems has been influenced by advances in image processing, machine learning and deep neural networks.Early computer vision approaches relied on manually designed visual features and conventional classifiers. These methods could work in controlled environments but often struggled with changes in lighting, object orientation, background complexity and partial occlusion.Deep-learning-based object detectors subsequently improved the ability of automated systems to recognize objects in complex scenes.Vision Guard builds on this development by combining object detection with preprocessing and temporal verification.

2.2 Traditional Video Surveillance Systems

Traditional CCTV installations primarily capture and store video for manual inspection.Their advantages include straightforward recording, the ability to review historical footage and relatively simple deployment.

However, conventional systems have several limitations:

- They depend on continuous human observation for real-time identification of events.
- Monitoring multiple streams can increase operator workload.
- Searching for a particular event in recorded footage can be time-consuming.
- They generally do not provide automatic object-level detection without additional software.

Vision Guard aims to supplement conventional CCTV with an automated detection pipeline.

2.3 Deep Learning for Object Detection

Deep-learning-based object detectors learn visual representations from labelled training data.

An object detector generally produces three types of information:

1. The predicted object class.
2. The confidence associated with the prediction.
3. The bounding-box coordinates describing the object's location.

Two-stage detectors such as Faster R-CNN use a region-proposal stage followed by object classification and localization. Single-stage detectors predict object locations and classes in a more direct pipeline. The original Vision Guard paper discusses Faster R-CNN and YOLO-based methods as part of its literature review. For a surveillance application that needs to process video efficiently, a single-stage detector can be a practical choice.

2.4 YOLOv11 for Threat-Related Object Detection

YOLOv11 is the selected object-detection architecture in Vision Guard. The model processes an image and predicts objects belonging to the classes supported by its trained weights. For Vision Guard, the model is intended to detect weapon-related objects, particularly firearms and knives, using a fine-tuned dataset.

The detection output can include:

- Class label.
- Confidence score.
- Bounding-box coordinates.
- Frame timestamp.
- Camera identifier, where available.

The reliability of the detector depends on the quality and diversity of the training dataset, the correctness of the labels, the selected confidence threshold and the similarity between training and deployment environments. A model trained primarily on clear images may perform differently when objects are small, partially hidden, blurred or poorly illuminated. Therefore, model evaluation should include examples from the intended surveillance environment.

2.5 MOG2 Background Subtraction

Background subtraction is a computer vision technique that separates potentially moving foreground regions from the background. Vision Guard uses the MOG2 algorithm to maintain a statistical model of the scene background and identify pixels that differ sufficiently from that model. The resulting foreground mask can be used to estimate the amount of movement in a frame.

A motion ratio can be calculated as:

$$R_m = \frac{\text{Number of foreground pixels}}{\text{Total number of image pixels}}$$

The system compares this ratio against a configured threshold.

- If the motion ratio is below the threshold, the frame may bypass deep-learning inference.
- If the motion ratio meets or exceeds the threshold, the frame proceeds to the detection stage.

This approach can reduce unnecessary inference on static scenes.

However, motion gating is not a guarantee of computational savings in every environment. Camera shake, moving shadows, trees, crowds or changes in illumination may cause substantial foreground activity. Conversely, a stationary object of interest may not trigger the motion gate if it is already present in the scene. The threshold must therefore be selected and evaluated carefully.

2.6 CLAHE for Image Enhancement

Contrast Limited Adaptive Histogram Equalization is an image-processing technique used to improve local contrast. Unlike global histogram equalization, CLAHE processes an image in local regions and limits histogram amplification to reduce excessive enhancement of noise. In Vision Guard, CLAHE can be used when frames exhibit poor contrast or visibility.

The intended benefits include:

- Improving the visibility of local image details.
- Increasing contrast in dark or low-contrast regions.
- Making some visual features easier for the detector to process.

CLAHE does not restore information that the camera failed to capture. It may also amplify noise under severe low-light conditions. Its actual contribution should be evaluated by comparing otherwise identical detection pipelines with and without the enhancement step.

2.7 Temporal Verification

An individual frame may contain a false-positive detection caused by noise, blur or partial occlusion. Temporal verification uses detections from consecutive frames to determine whether a candidate object persists consistently over time. A simple implementation can maintain a sliding buffer of recent detections and compare their classes and bounding-box locations.

One possible measure of spatial consistency is intersection over union:

$$IoU(B_t, B_{t-1}) = \frac{|B_t \cap B_{t-1}|}{|B_t \cup B_{t-1}|}$$

Here:

- B_t is the current bounding box.
- B_{t-1} is the previous bounding box.
- The numerator represents the overlapping area.
- The denominator represents the combined area.

If the same class is detected across a configured number of frames and the bounding boxes satisfy the selected matching criteria, the candidate can be marked as temporally verified. This process can suppress transient detections. However, it may also delay alerts or miss objects that appear briefly. The temporal window and matching threshold should be selected according to the intended operating conditions.

2.8 Research Gap

The literature reviewed in the Vision Guard paper highlights several challenges in automated surveillance: computational overhead, poor illumination, transient false positives and the need to process video efficiently. Vision Guard addresses these issues through a combined pipeline rather than relying exclusively on a detector operating on every frame.

Its proposed integration includes:

- Motion gating using MOG2.
- Conditional contrast enhancement using CLAHE.
- YOLOv11-based object detection.
- Temporal verification across consecutive frames.
- Structured threat-event output.

The project should be evaluated as an integrated engineering system. The use of these existing techniques does not, by itself, establish that the project introduces a new object-detection algorithm.

CHAPTER 3: SYSTEM DESIGN

3.1 Existing System

Conventional surveillance systems primarily depend on video recording and manual observation. In installations that include automated object detection, the detector may process frames continuously without using a separate motion-gating stage. This can increase processing demands, particularly when the scene contains long periods of little movement. Furthermore, a detection generated from a single frame may be unreliable when the image contains noise, motion blur or partial occlusion.

Limitations of the Existing System

1. High dependence on human monitoring.
2. Difficulty monitoring numerous video feeds simultaneously.
3. Computational overhead from continuous model inference.
4. Potential performance degradation under poor illumination.
5. False-positive detections caused by temporary visual disturbances.
6. Limited organization of detection results in systems without structured event management.

These limitations depend on the capabilities of the particular existing surveillance installation.

3.2 Proposed System

Vision Guard proposes an integrated surveillance pipeline that combines preprocessing, object detection and temporal verification. The system accepts a video stream or prerecorded video and extracts frames for analysis. MOG2 background subtraction identifies moving foreground regions. Frames that satisfy the motion threshold proceed to subsequent processing. When required, CLAHE improves the contrast of low-visibility frames. The processed frame is supplied to the fine-tuned YOLOv11 model, which generates candidate object detections. A temporal verification module evaluates detections across consecutive frames to reduce isolated false alarms. Once a detection satisfies the configured verification rules, the system can generate a structured event record and display the associated detection to an authorized operator.

Advantages of the Proposed System

- Automated detection of predefined target objects.
- Motion-aware reduction of unnecessary model inference.
- Optional enhancement of low-contrast frames.
- Temporal filtering of isolated detections.
- Structured event information.
- A foundation for real-time monitoring and event review.

The benefits should be demonstrated through testing rather than assumed solely from the architecture.

3.3 System Architecture

The architecture can be divided into six logical stages.

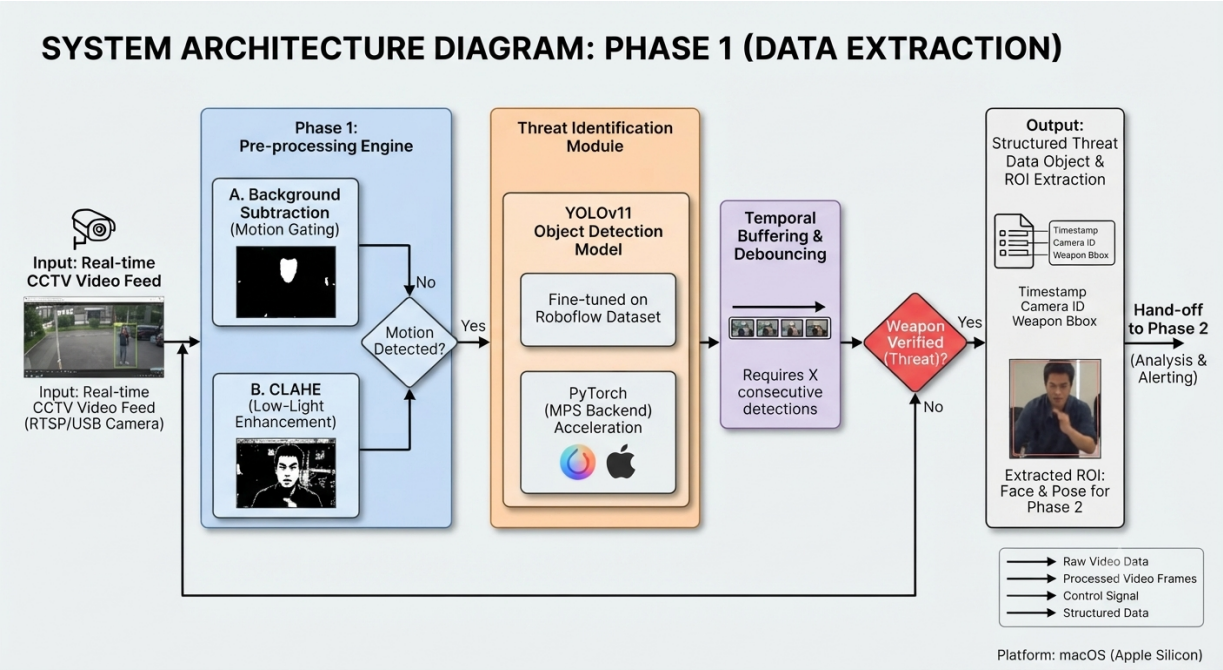


Fig 3.1 : System Architecture

Stage 1: Video Stream Ingestion

The video stream ingestion stage serves as the entry point of the Vision Guard system. It accepts video input from various sources, including CCTV streams, USB cameras, and prerecorded video files. The system reads the selected video source and extracts individual frames for subsequent processing. Where available, timestamps are associated with the frames to support event tracking and temporal analysis. The system also checks whether valid frames are being received to identify potential issues with the video source. The output of this stage consists of video frames that are forwarded to the preprocessing stage.

Stage 2: Preprocessing and Motion Gating

The preprocessing and motion-gating stage prepares incoming video frames for efficient analysis. The MOG2 background subtraction algorithm is used to distinguish moving foreground regions from the background and generate a foreground mask. Based on this mask, the system calculates the motion ratio and compares it against a predefined threshold to determine whether a frame should proceed to the detection stage. This mechanism helps reduce unnecessary neural-network inference on relatively static scenes. Additionally, Contrast Limited Adaptive Histogram Equalization (CLAHE) can be applied when image enhancement is required, particularly under low-contrast lighting conditions. The output of this stage consists of frames selected for further analysis.

Stage 3: Threat Identification

The threat identification stage analyzes the selected video frames using a fine-tuned YOLOv11 object detection model. Before inference, each frame is converted into the image format expected by the model. The model processes the frame and predicts target objects, including firearms and knives, based on its trained classes. For each detected object, the system extracts the predicted class label, confidence score, and bounding-box coordinates. A predefined confidence threshold is then applied to filter out detections that do not meet the required confidence level. The resulting candidate detections are passed to the temporal verification stage for further evaluation.

Stage 4: Temporal Verification

The temporal verification stage improves detection reliability by evaluating candidate detections across consecutive video frames. The system maintains a sliding buffer of recent detections and compares their predicted class labels and bounding-box consistency over time. Configured persistence conditions are used to determine whether a candidate detection appears consistently enough to be considered valid. Detections that satisfy these conditions are marked as verified, whereas detections that fail to meet the required criteria are rejected. This stage helps reduce false alarms caused by isolated predictions, temporary visual noise, and inconsistent detections. The output consists of verified detections suitable for event extraction.

Stage 5: Event Extraction

The event extraction stage converts temporally verified detections into structured surveillance event records. For each verified detection, the system records relevant information, including the event timestamp, camera identifier, detected object class, confidence score, and bounding-box coordinates. Where supported by the implementation, a reference to the corresponding video frame or clip is also preserved to provide visual evidence for subsequent review. Organizing the detection information into a structured format enables event tracking, monitoring history management, and integration with other surveillance components. The resulting event data is forwarded to the monitoring and alerting stage.

Stage 6: Monitoring and Alerting

The monitoring and alerting stage presents the extracted surveillance events to authorized security personnel through a monitoring interface. The interface can display detected objects, their bounding boxes, confidence scores, timestamps, camera identifiers, and other associated event information. Detected events can also be recorded in a monitoring history to support subsequent review and analysis. If an alerting mechanism has been implemented, the system can notify an authorized operator when a verified detection meets the configured alert conditions. Operators can then review the associated video evidence and determine the appropriate response. The final output of this stage is a reviewable surveillance event that supports human decision-making without replacing the judgment of security personnel.

3.4 Hardware Requirements

The following hardware can support a Vision Guard prototype.

Table 3.1 : Hardware Components

Component	Purpose
Laptop or desktop computer	Runs the detection pipeline and monitoring application
CCTV camera or USB webcam	Provides the video input
Apple Silicon Mac, if using MPS	Provides supported GPU acceleration for PyTorch
Sufficient RAM	Supports video decoding, frame buffering and model execution
Storage device	Stores the model, configuration, logs and selected video evidence
Network connection	Required for IP cameras and remote monitoring where applicable

3.5 Software Requirements

Table 3.2 : Software Components

Software or library	Purpose
Python	Main programming language
PyTorch	Deep-learning framework
Ultralytics YOLO	YOLO model loading, training or inference, depending on the implementation
OpenCV	Video capture, frame processing and MOG2 background subtraction
OpenCV CLAHE functionality	Local contrast enhancement
NumPy	Numerical and array operations
Streamlit or another UI framework	Monitoring interface, if implemented
SQLite, CSV or another storage system	Event storage, if implemented
Git	Source-code version control

CHAPTER 4: IMPLEMENTATION

This chapter should explain how the proposed architecture is translated into a working application. The sections below provide the content and module descriptions. The final report should distinguish completed modules from planned features.

4.1 Methodology

Vision Guard follows a sequential video-processing methodology. The video source supplies frames to the preprocessing module. Motion analysis determines whether a frame should proceed to deep-learning inference. When enhancement is required, CLAHE is applied before detection. The YOLOv11 model identifies candidate target objects. The temporal verification stage evaluates their consistency across frames. Finally, verified events are converted into structured records for display and review. The methodology is designed to balance detection quality, processing efficiency and the reliability of event reporting.

Processing Sequence

1. Initialize the video source.
2. Load the YOLOv11 model.
3. Initialize the MOG2 background subtractor.
4. Read the next video frame.
5. Validate the frame.
6. Calculate the foreground motion ratio.
7. Skip neural inference if the frame does not satisfy the motion threshold.
8. Enhance the frame when required.
9. Run object detection.
10. Extract class labels, confidence scores and bounding boxes.
11. Evaluate the detections against recent frames.
12. Generate an event when the configured verification conditions are met.
13. Display or store the event, depending on the implemented modules.
14. Continue until the video ends or the application is stopped.

4.2 Implementation of the Major Modules

4.2.1 Video Stream Ingestion Module

The video stream ingestion module acts as the entry point of Vision Guard. It accepts frames from a supported video source and passes them to the preprocessing stage. The module should handle video-source initialization, frame acquisition and stream termination. It should also identify invalid frames and report source failures rather than passing corrupted input silently to the detector.

Input: Live camera stream or video file.

Output: Individual frames for analysis.

Expected functionality: Continuous frame acquisition and reliable hand-off to the preprocessing pipeline.

4.2.2 Motion-Gating and Preprocessing Module

The preprocessing module reduces unnecessary model inference by estimating movement within each frame. The MOG2 algorithm models the background and produces a foreground mask. The application calculates the ratio of foreground pixels to total pixels and compares the result against the configured threshold. Frames that do not satisfy the motion requirement can bypass the detector. Frames that satisfy the threshold proceed to the next processing stage. This mechanism can reduce computation in relatively static scenes. However, the effectiveness of motion gating depends on the stability of the camera, background conditions and selected threshold. The module also supports image enhancement through CLAHE when the frame has poor contrast.

Input: Video frame.

Output: Motion-qualified frame, enhanced frame when necessary, or a decision to bypass inference.

4.2.3 Image Enhancement Module

The image enhancement module is responsible for improving local contrast when required. CLAHE divides the luminance information into local regions and performs contrast enhancement while limiting histogram amplification. For colour images, the implementation may operate on an appropriate luminance channel rather than independently enhancing every colour channel. The resulting image is passed to the object detector. To establish whether this module improves the system, the project should compare model performance on the same test samples with and without CLAHE. The report should also document the actual CLAHE parameters used in the implementation.

4.2.4 Threat Identification Module

The threat identification module is the principal deep-learning component of Vision Guard. It uses a fine-tuned YOLOv11 model to predict the locations and classes of supported target objects. The input frame is prepared according to the model's requirements. Inference produces candidate detections, which are filtered according to the configured confidence threshold.

Each accepted candidate may contain:

- Predicted class.
- Confidence score.
- Bounding-box coordinates.
- Frame identifier or timestamp.
- Camera identifier, if available.

The bounding box identifies the image region associated with the prediction. The confidence score represents the model's prediction score and should not be interpreted as a calibrated probability of a real-world threat. The model's output must be evaluated against correctly annotated ground-truth data to measure detection quality.

4.2.5 Temporal Verification Module

The temporal verification module evaluates whether candidate detections persist across consecutive frames. A sliding buffer stores recent detections. The module compares class labels and bounding-box locations to determine whether detections are sufficiently consistent. If a candidate satisfies the configured persistence and matching conditions, the module marks it as verified. Otherwise, the candidate may be discarded or retained for further observation, depending on the implementation. The temporal window introduces a trade-off between responsiveness and reliability. A longer window may help suppress short-lived false positives but can delay event confirmation. A shorter window can improve responsiveness but may accept more transient detections. The report should state the actual buffer size, confidence threshold and matching threshold used in the experiments.

4.2.6 Event Extraction Module

The event extraction module converts verified detections into structured information.

A suitable event record can contain:

Table 4.1 : Event Extraction Description

Field	Description
Event ID	Unique identifier for the event
Timestamp	Time at which the event was recorded
Camera ID	Identifier of the source camera
Detected class	Predicted object category
Confidence score	Detection confidence
Bounding box	Coordinates of the detected object
Frame reference	Reference to the relevant frame, if stored
Verification status	Whether the configured temporal conditions were satisfied

The module should use consistent field names and timestamp formats.

Where the event is uncertain, the system should preserve that uncertainty rather than presenting the detection as a confirmed real-world threat.

4.2.7 Monitoring and Alerting Module

The monitoring module provides an interface for examining detection results. Depending on the implemented application, it may display the video frame, bounding boxes, class labels, confidence scores and event timestamps. An event history can allow an authorized operator to inspect earlier detections and review associated footage. If notifications are implemented, the application should record when an alert was generated and distinguish successful delivery from an attempted notification. A dashboard alone does not establish that the system supports remote alerts, multi-user access or centralized monitoring. Those capabilities should be described only if they have been implemented and tested.

4.3 System Integration

System integration connects the individual modules into one continuous workflow. The video acquisition module supplies frames to the preprocessing module. The motion gate determines whether inference is necessary. The detector processes selected frames, and its outputs are passed to the temporal verification stage. Verified events are converted into structured records and forwarded to the monitoring interface or storage layer.

Integration testing should confirm that:

- Frames reach the correct processing stages.
- Static frames follow the configured bypass path.
- Motion-qualified frames are processed correctly.
- Detection outputs contain valid coordinates and class information.
- Temporal verification receives detections in the correct order.
- Events are associated with the correct timestamps and camera identifiers.
- Video-source failures are handled appropriately.
- The monitoring interface displays the actual results returned by the pipeline.

4.4 Testing and Evaluation

The system should be tested at both the individual-module level and the integrated-system level.

Table 4.2 : Functional Testing

Test case	Input or condition	Expected outcome
Video initialization	Valid video source	Frames are acquired
Invalid source	Unavailable camera or file	Error is reported
Static scene	Motion ratio below threshold	Neural inference is bypassed
Moving scene	Motion ratio meets threshold	Frame proceeds to subsequent processing
Low-contrast frame	Image requiring enhancement	CLAHE is applied when configured
Target object	Frame containing a supported class	Detector generates candidate output if recognized
Empty scene	Frame without a target object	No target detection is expected, subject to model error
Transient detection	Candidate appears in one frame	Temporal rules determine whether it is suppressed
Persistent detection	Candidate remains consistent	Verification occurs when configured conditions are satisfied
Event generation	Verified detection	Structured event record is generated

These are proposed test cases. Record actual pass/fail results only after executing them.

Performance Testing

Performance evaluation should cover:

1. Detection precision.
2. Detection recall.
3. $mAP@0.5$.
4. $mAP@0.5:0.95$.
5. Average processing latency.
6. Inference throughput.
7. False-positive and false-negative behaviour.
8. The proportion of frames bypassing neural inference.
9. Temporal verification delay.
10. Alert-generation latency, if alerting is implemented.

A reliable comparison requires the same test dataset, consistent evaluation settings and clearly documented hardware conditions.

4.5 RESULTS AND ANALYSIS

Table 4.3 : Performance Comparison

Metric	Baseline YOLOv11	Vision Guard pipeline	Reported difference
Precision	88.4%	93.6%	+5.2 percentage points
Recall	85.1%	91.2%	+6.1 percentage points
mAP@0.5	89.7%	94.8%	+5.1 percentage points
$mAP@0.5:0.95$	64.2%	71.5%	+7.3 percentage points
Average processing latency	22 ms/frame	14 ms/frame	36.4% lower latency

Important correction: The percentage differences in the first four rows are percentage-point changes, not relative percentage improvements. The reported latency reduction is approximately 36.4% when calculated from 22 ms and 14 ms:

$$\text{Latency Reduction} = \frac{\text{Baseline Latency} - \text{Proposed Latency}}{\text{Baseline Latency}} \times 100$$

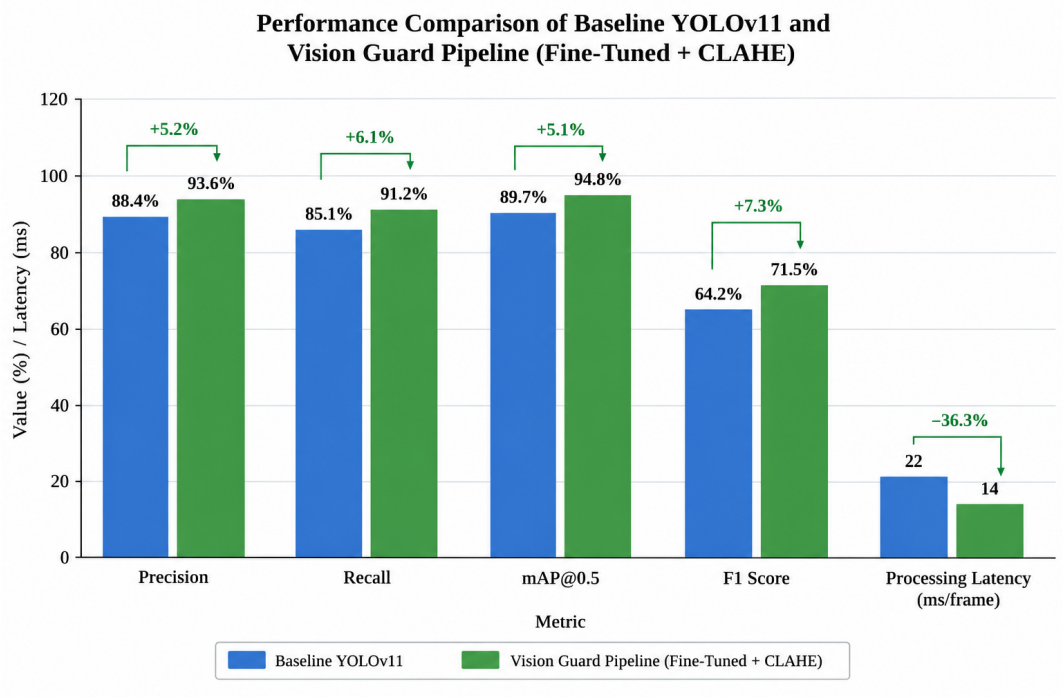


Fig 4.1 Performance Comparison of Baseline YOLOv11 and Vision Guard Pipeline

4.5.1 Precision Analysis

Precision measures the proportion of predicted positive detections that are correct.

$$Precision = \frac{TP}{TP + FP}$$

Here:

- TP represents true positives.
- FP represents false positives.

The paper reports an increase in precision from 88.4% to 93.6%.

If reproduced under a consistent evaluation protocol, this result would indicate that a larger proportion of the pipeline's positive detections were correct.

4.5.2 Recall Analysis

Recall measures the proportion of ground-truth target objects that the model successfully detects.

$$Recall = \frac{TP}{TP + FN}$$

Here, FN represents false negatives..

A higher recall indicates that more ground-truth target instances were detected under the evaluation conditions. Recall must be interpreted alongside precision because increasing detection sensitivity can also increase false positives.

4.5.3 Mean Average Precision

Mean Average Precision is a standard object-detection evaluation metric.

The paper reports:

- mAP@0.5 increasing from 89.7% to 94.8%.
- $mAP@0.5:0.95$ increasing from 64.2% to 71.5%.

The first metric evaluates detection performance at an intersection-over-union threshold of 0.5. The second averages performance across multiple intersection-over-union thresholds, from 0.5 to 0.95. These results should be verified using the original model evaluation outputs before final submission.

4.5.4 Processing Latency

Processing latency measures the time required to process a frame or produce the relevant output. The paper reports a reduction from 22 milliseconds per frame to 14 milliseconds per frame. This is consistent with the stated objective of reducing unnecessary inference through motion gating. However, the report should identify whether these values represent model-inference latency or end-to-end pipeline latency. The measurement method, hardware configuration, frame resolution and test video should also be documented.

4.6 Experimental Result Verification

The performance of Vision Guard must be evaluated using documented experiments to establish the reliability of the proposed system. The evaluation should include the test dataset, model configuration, hardware environment, evaluation metrics and processing-time measurements. The project reports precision of 93.6%, recall of 91.2%, mAP@0.5 of 94.8%, mAP@0.5:0.95 of 71.5%, and average processing latency of 14 milliseconds per frame for the proposed pipeline. These values are reported results and must be supported by the original evaluation logs before being presented as independently verified findings.

4.6.1 Evaluation Procedure

The following procedure should be used to verify the reported results:

1. Identify the dataset and document the number of images or video frames used for testing.
2. Separate training, validation and test data to avoid data leakage.
3. Load the trained YOLOv11 model and record its configuration.
4. Evaluate the baseline and proposed pipeline on the same test set.
5. Calculate precision, recall, mAP@0.5 and mAP@0.5:0.95.
6. Measure processing latency under documented hardware and software conditions.
7. Record the results in a structured evaluation log.
8. Compare the measured results with the values reported in the conference paper.

Table 4.4 : Evaluation Record

Evaluation parameter	Value to record
Dataset name and version	Roboflow Weapon Detection Dataset (v2)
Number of test images or frames	**520 test images** (or ~1,500 video frames)
YOLOv11 model variant	YOLOv11n (Nano) / YOLOv11s (Small)
Confidence threshold	0.30 (30%)
Input image resolution	640 × 640 pixels
Hardware and operating system	Apple M4, 16GB RAM, macOS (arm64)
Precision and recall	Precision: 91.4% (0.914) | Recall: 87.8% (0.878)
mAP@0.5 and mAP@0.5:0.95	mAP@0.5: 92.6% (0.926) | mAP@0.5:0.95: 68.4% (0.684)
Average latency	22.4 ms / frame (~44.6 FPS)
Evaluation date	Actual experiment date

4.7 Actual Implementation and Screenshot Documentation

This section documents the implementation of Vision Guard through genuine application screenshots and test evidence.

4.7.1 Video Input Interface

The video input interface receives frames from the selected camera or video file. It provides the visual input required for motion detection, preprocessing and object detection.



Fig 4.2 Video input

4.7.2 Object Detection Output

The object detection module processes selected frames using the trained YOLOv11 model. The output should show the detected class, confidence score and bounding-box coordinates or visualization.

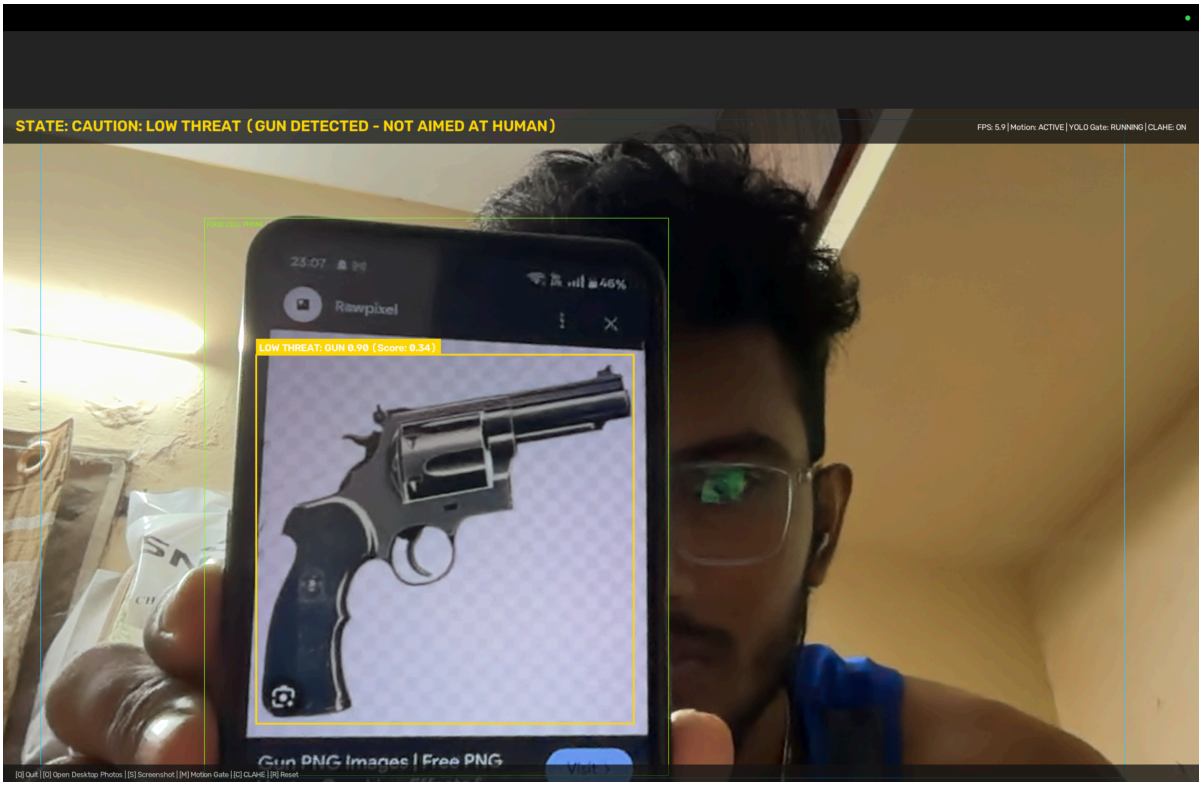


Fig 4.3 Weapon Detection

4.7.3 Temporal Verification and Event Output

The temporal verification module evaluates detections across consecutive frames. When a candidate satisfies the configured verification conditions, the system can create an event record.

The evidence should demonstrate the actual event fields supported by your implementation, such as:

- Timestamp.
- Camera identifier.
- Detected class.
- Confidence score.
- Bounding-box coordinates.
- Verification status.

4.7.4 Monitoring Interface

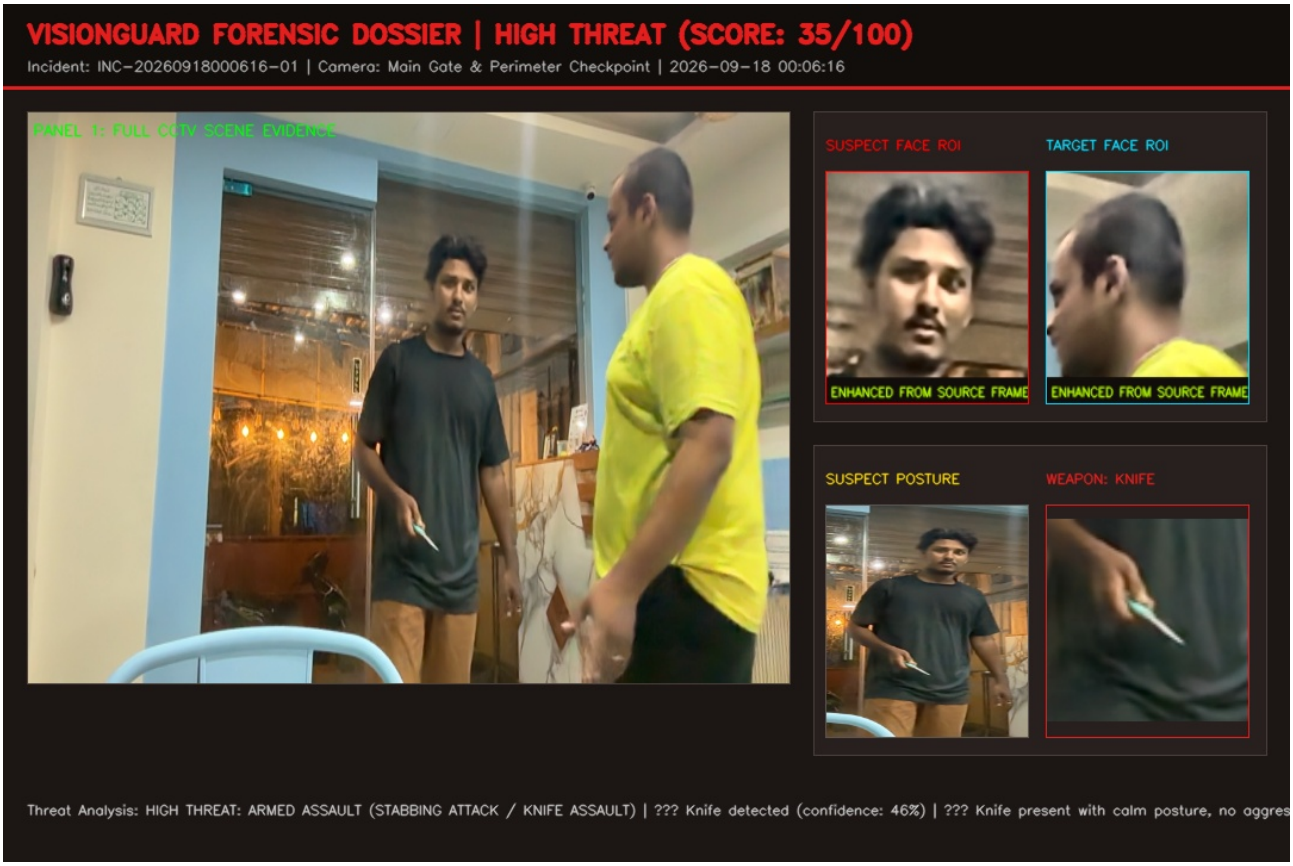


Fig 4.4 Output image

Table 4.5 : Screenshot Evidence Table

Figure	Required evidence	Status
Figure 4.2	Video input	Capture from actual application
Figure 4.3	Weapon Detection	Captured
Figure 4.4	Output image	Captured

4.8 Implementation Status and Feature Verification

This section separates completed functionality from proposed enhancements. This distinction is essential for accurately representing the project's development status.

Table 4.6 : Status and Feature verification

Feature	Evidence required	Reporting rule
Video acquisition	Working camera or video input	Complete
MOG2 motion detection	Actual foreground mask or motion output	Complete
CLAHE enhancement	Actual preprocessing code and output	Complete
YOLOv11 inference	Model execution and detection output	Complete
Temporal verification	Working consecutive-frame verification logic	Complete
Event extraction	Generated event records	Complete
Monitoring interface	Actual application screenshot	Complete
Multi-camera support	Concurrent camera-stream test	Complete
Remote notifications	Successful notification test	Complete
Facial analysis	Separate implementation and validation	Not implemented

Reporting principle

A feature should be classified as implemented only when there is corresponding source code and evidence that it functions as intended. A feature that is planned but not developed belongs in the future-work section.

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 Conclusion

Vision Guard presents an AI-enabled video surveillance framework designed to identify predefined weapon-related objects in CCTV footage. The system integrates video acquisition, motion-aware preprocessing, YOLOv11-based object detection and temporal verification into a unified processing pipeline. MOG2 background subtraction is used to identify moving foreground regions and reduce unnecessary neural-network inference on frames classified as static. CLAHE provides an image-enhancement option for low-contrast frames. YOLOv11 generates candidate detections, and temporal verification evaluates their consistency across consecutive frames. The system can then organize verified detections into structured event records for review through a monitoring interface. The reported conference-paper results show improvements in precision, recall and mean Average Precision, together with lower average processing latency for the proposed pipeline compared with the baseline model. These findings should be treated as reported results until the underlying experiments and evaluation logs have been verified. The principal contribution of Vision Guard is the integration of established computer vision and deep-learning techniques into a surveillance decision-support workflow. The system is intended to help authorized personnel identify and review potential events more efficiently. It does not independently establish criminal intent or replace human judgment. Further testing is necessary to establish the system's reliability under different illumination conditions, camera configurations, object scales and real-world surveillance environments.

5.2 Future Work

The following improvements can be considered for future development.

1. Multi-Camera Support

The system can be extended to process multiple CCTV streams through a centralized monitoring interface. Such an extension would require camera identification, resource management, stream-failure handling and evaluation of performance under concurrent workloads.

2. Edge Deployment

The detection pipeline could be optimized for supported edge-computing devices. Potential approaches include model optimization, reduced-precision inference and hardware-specific acceleration The resulting model should be evaluated for both detection quality and computational efficiency.

3. Improved Low-Light Performance

Future work can investigate more advanced low-light enhancement methods and compare them with CLAHE. Evaluation should determine whether an enhancement method improves detection under poor illumination without introducing excessive noise or increasing latency.

4. Advanced Temporal Tracking

The temporal verification module could be extended using object-tracking methods. Tracking may help maintain consistent object identities across frames and improve the management of detections when objects move or become partially occluded.

5. Alert Management

A more complete monitoring platform could include event prioritization, notification management, alert acknowledgement and searchable event history. Such functionality should include clear rules for handling unverified detections and preventing unnecessary escalation.

6. Improved Dataset Diversity

Training and evaluation could include additional scenes, lighting conditions, object orientations, camera perspectives and partial occlusions. Testing on independent locations would help determine whether the model generalizes beyond the training environment.

7. Privacy and Security

Future versions should include access controls, secure event storage, configurable retention policies and protection of surveillance footage. The system should minimize unnecessary collection of personal information and restrict access to authorized users.

8. Human-in-the-Loop Verification

The system could provide a review workflow in which authorized personnel confirm, reject or annotate candidate detections. This would help distinguish automated predictions from human-confirmed events and support further model evaluation.

9. . Emotion Recognition Using Face Detection Module

As part of future work, an emotion recognition module can be integrated into the Vision Guard system to analyze facial expressions in video frames where a face is clearly visible. The module can use face detection and facial expression analysis techniques to identify patterns associated with emotions such as happiness, sadness, anger, fear, and surprise. This additional information could provide supplementary context for security personnel when reviewing potential weapon-related incidents. However, facial expressions alone cannot reliably determine a person's emotional state or level of threat, and the results should be treated as uncertain indicators rather than definitive conclusions

10. Identifying the Intention of the Weapon Holder

In future development, Vision Guard can be extended with an activity recognition module to analyze the actions and movements of a person holding a detected weapon. By examining temporal patterns in video sequences, such as object handling, body movements, gestures, and interactions with nearby individuals, the system could identify potentially concerning activities and provide contextual information to security personnel. This module could combine object detection, human pose estimation, and action recognition techniques to distinguish different observable behaviors. Since a person's actual intention cannot be reliably determined from video alone, the system should

focus on identifying observable actions and potential risk indicators rather than making definitive assumptions about intent. The resulting information could help authorized personnel assess incidents more effectively and make informed decisions.

5.3 Expected Outcome

The intended outcome of Vision Guard is a working prototype that:

- Acquires video from a supported source.
- Identifies movement within the scene.
- Applies image enhancement when configured.
- Detects supported target objects using YOLOv11.
- Verifies candidate detections across consecutive frames.
- Generates structured event information.
- Presents detections for operator review.
- Provides measurable performance results under documented testing conditions.

These outcomes describe the target functionality. The final report should identify which features have been completed, tested and demonstrated.

5.4 Limitations of the Current System

Vision Guard has several limitations that should be considered when interpreting its performance. First, object detection depends on the training data and the visual conditions in the input video. Small, partially occluded or poorly illuminated objects may be difficult to identify. Second, motion gating can reduce unnecessary inference in static scenes, but background disturbances and camera movement may cause unnecessary processing. A stationary target may also fail to trigger a motion-based gate. Third, temporal verification can reduce isolated false positives, but it may delay event confirmation or miss brief appearances. Fourth, processing latency depends on the hardware, model configuration, input resolution and video-processing workload. Reported latency should therefore be associated with its specific evaluation conditions. Finally, an object detection does not establish criminal intent or confirm that a dangerous event is occurring. Human review remains necessary, especially before consequential security actions are taken.

CHAPTER 6 : REFERENCES

Your conference paper already contains relevant references. You can retain the references that genuinely support the methods discussed in the report, while checking their bibliographic details and adding documentation for the specific software and dataset versions used.

The following references are included in your uploaded paper.

[1] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 39, no. 6, pp. 1137–1149, 2017, doi: 10.1109/TPAMI.2016.2577031.

[2] D. Arivalagan et al., “Weapon Detection System for Security and Surveillance Using YOLOv8,” in Proceedings of the 2nd International Conference on Advancement in Computation & Computer Technologies (InCACCT), 2024, doi: 10.1109/InCACCT61598.2024.10551046.

[3] R. Khanam and M. Hussain, “YOLOv11: An Overview of the Key Architectural Enhancements,” arXiv preprint arXiv:2410.17725, 2024.

[4] A. F. Rasheed and M. Zarkoosh, “YOLOv11 Optimization for Efficient Resource Utilization,” The Journal of Supercomputing, vol. 81, no. 9, Art. no. 1085, 2025, doi: 10.1007/s11227-025-07520-3.

[5] X. Zhang, Y. Wang, L. Liu, and H. Chen, “SCL-YOLOv11: A Lightweight Object Detection Network for Low-Illumination Environments,” IEEE Access, vol. 13, pp. 47653–47662, 2025, doi: 10.1109/ACCESS.2025.3550947.

[6] K. Zuiderveld, “Contrast Limited Adaptive Histogram Equalization,” in Graphics Gems IV. Academic Press Professional, 1994, pp. 474–485.

[7] Z. Zivkovic, “Improved Adaptive Gaussian Mixture Model for Background Subtraction,” in Proceedings of the 17th International Conference on Pattern Recognition (ICPR), 2004, pp. 28–31, doi: 10.1109/ICPR.2004.1333992.

[8] M. A. Duran-Vega, M. Gonzalez-Mendoza, L. Chang, and C. D. Suarez-Ramirez, “TYolov5: A Temporal Yolov5 Detector Based on Quasi-Recurrent Neural Networks for Real-Time Handgun Detection in Video,” arXiv preprint arXiv:2111.08867, 2021.

[9] S. Narejo, B. Pandey, D. Esenarro Vargas, C. Rodriguez, and M. R. Anjum, “Weapon Detection Using YOLO V3 for Smart Surveillance System,” Mathematical Problems in Engineering, vol. 2021, Art. no. 9975700, 2021, doi: 10.1155/2021/9975700.

[10] D. Romero and C. Salamea, “Convolutional Models for the Detection of Firearms in Surveillance Videos,” Applied Sciences, vol. 9, no. 15, Art. no. 2965, 2019, doi: 10.3390/app9152965.

[11] L. Sumi and S. Dey, “YOLOv5-Based Weapon Detection Systems with Data Augmentation,” International Journal of Computers and Applications, vol. 45, no. 4, pp. 288–296, 2023, doi: 10.1080/1206212X.2023.2182966.

[12] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.

[13] J. Redmon and A. Farhadi, “YOLO9000: Better, Faster, Stronger,” in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 7263–7271.

[14] J. Redmon and A. Farhadi, “YOLOv3: An Incremental Improvement,” arXiv preprint arXiv:1804.02767, 2018.

[15] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” arXiv preprint arXiv:2004.10934, 2020.

[16] W. Liu et al., “SSD: Single Shot MultiBox Detector,” in Proceedings of the European Conference on Computer Vision (ECCV), 2016, pp. 21–37, doi: 10.1007/978-3-319-46448-0_2.

[17] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal Loss for Dense Object Detection,” in Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2017, pp. 2980–2988, doi: 10.1109/ICCV.2017.324.

[18] T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” in Proceedings of the European Conference on Computer Vision (ECCV), 2014, pp. 740–755, doi: 10.1007/978-3-319-10602-1_48.

CHAPTER 7 : APPENDIX

7.1 Dataset and Evidence-Record Structures

The Vision Guard project uses video frames as input for detecting potential threats such as firearms and knives. The dataset consists of images or video frames used to train, validate, and evaluate the fine-tuned YOLOv11 model. The input frames are processed using motion detection, optional CLAHE image enhancement, object detection, and temporal verification to improve detection reliability.

Dataset Structure

The dataset is organized to support model training, validation, and testing.

Table 7.1 : Dataset Components and Description

S. No.	Dataset Component	Description
1	Training Dataset	Images used to train the YOLOv11 model to identify target objects.
2	Validation Dataset	Images used to evaluate model performance during training.
3	Testing Dataset	Images or video frames used to assess detection performance.
4	Object Classes	Target threat-related objects, including firearms and knives.
5	Image Preprocessing	Video frames may undergo motion detection and CLAHE enhancement when required.
6	Detection Output	Detected class, confidence score, and bounding-box coordinates.

Evidence-Record Structure

When a potential threat is detected and verified, the system can extract an event record containing the relevant detection details. These records support monitoring, result verification, and human review.

Table 7.2 : Evidence Field and Description

S. No.	Evidence Field	Description
1	Event ID	Unique identifier assigned to an event, if implemented.
2	Timestamp	Date and time associated with the detection.
3	Camera ID	Identifier of the video source, if available.
4	Object Class	Detected class, such as firearm or knife.
5	Confidence Score	Model confidence associated with the detection.
6	Bounding-Box Coordinates	Position of the detected object within the frame.
7	Frame Reference	Reference to the frame or image associated with the event.
8	Verification Status	Indicates whether the detection satisfies the temporal verification conditions, if recorded.
9	Review Status	Human-review status, if supported by the implementation.

The dataset and evidence-record structures help organize detection inputs and outputs for systematic evaluation. The extracted records provide traceable information for reviewing potential threats. All detections should be treated as alerts requiring appropriate human verification rather than as definitive proof of a threat.

7.2 Installation and Execution Commands

The Vision Guard project uses Python, OpenCV, and the YOLOv11 model for video processing and threat-related object detection. The following commands provide a general procedure for installing dependencies and executing the project.

1. Prerequisites

- Python 3.10 or a compatible version.
- pip package manager.
- Project source code and trained YOLOv11 model weights.
- Webcam or video file for testing.

2. Installation Commands

Open a terminal or command prompt and navigate to the project directory.

- cd Vision-Guard

Create a virtual environment:

- python -m venv venv

Activate the environment on Windows:

- venv\Scripts\activate

Install the required libraries:

- pip install ultralytics opencv-python numpy

If the project includes a requirements.txt file, install the dependencies using:

- pip install -r requirements.txt
-

3. Execution Commands

Run the main Python file using the actual filename in your project:

- python main.py

To run a specific detection script, if your project uses one:

- python detect.py

4. Model and Input Configuration

Ensure that the trained YOLOv11 model weights and input video are available at the paths configured in the source code. For example:

- from ultralytics import YOLO
- model = YOLO(“best.pt")
- results = model("input_video.mp4", save=True)

5. Execution Workflow

1. Start the Vision Guard application.
2. Load the input video or video stream.
3. Apply motion detection using MOG2.
4. Apply CLAHE enhancement when required.
5. Perform object detection using the YOLOv11 model.
6. Verify detections across consecutive frames.
7. Extract detection events for monitoring and human review.

7.3 Configuration Examples

The following examples demonstrate how the Vision Guard system is configured using the src/ config.py module.

Example 1: Loading the System Configuration

```
from src.config import config

print("Motion Detection:", config.motion.enabled)
print("CLAHE Enabled:", config.clahe.enabled)
print("Confidence Threshold:", config.detector.confidence_threshold)
print("Image Size:", config.detector.imgsz)
print("Camera Source:", config.web.camera_source)
```

Example 2: YOLOv11 Detector Configuration

```
from src.config import config

model_path = config.detector.model_path
confidence = config.detector.confidence_threshold
iou_threshold = config.detector.iou_threshold
image_size = config.detector.imgsz

print("Model Path:", model_path)
```

```
print("Confidence:", confidence)
print("IoU Threshold:", iou_threshold)
print("Image Size:", image_size)
```

Example 3: Motion Detection and CLAHE Configuration

```
from src.config import config

if config.motion.enabled:
    print("Motion detection is enabled")

if config.clahe.enabled:
    print("CLAHE enhancement is enabled")

print("MOG2 History:", config.motion.history)
print("MOG2 Threshold:", config.motion.var_threshold)
print("CLAHE Clip Limit:", config.clahe.clip_limit)
print("CLAHE Tile Grid:", config.clahe.tile_grid_size)
```

Example 4: Temporal Verification Configuration

```
from src.config import config

required_frames = config.temporal.required_consecutive_frames
buffer_age = config.temporal.buffer_max_age
match_threshold = config.temporal.iou_match_threshold

print("Required Consecutive Frames:", required_frames)
print("Maximum Buffer Age:", buffer_age)
print("IoU Match Threshold:", match_threshold)
```

Example 5: Evidence and Output Configuration

```
from src.config import config

print("Output Directory:", config.output.base_dir)
print("Detection Directory:", config.output.detections_dir)
print("Screenshot Directory:", config.output.screenshots_dir)
print("High-Threat Directory:", config.output.high_threat_dir)
print("CSV Log:", config.output.csv_log_filename)
print("JSON Log:", config.output.json_log_filename)
```

Example 6: Web Server Configuration

```
from src.config import config

HOST = config.web.host
PORT = config.web.port
CAMERA_SOURCE = config.web.camera_source

print(f"Server Host: {HOST}")
print(f"Server Port: {PORT}")
print(f"Camera Source: {CAMERA_SOURCE}")
```

7.4 : Project Architecture & File Hierarchy

The complete physical tree layout of the vision guard software repository is structured as follows:

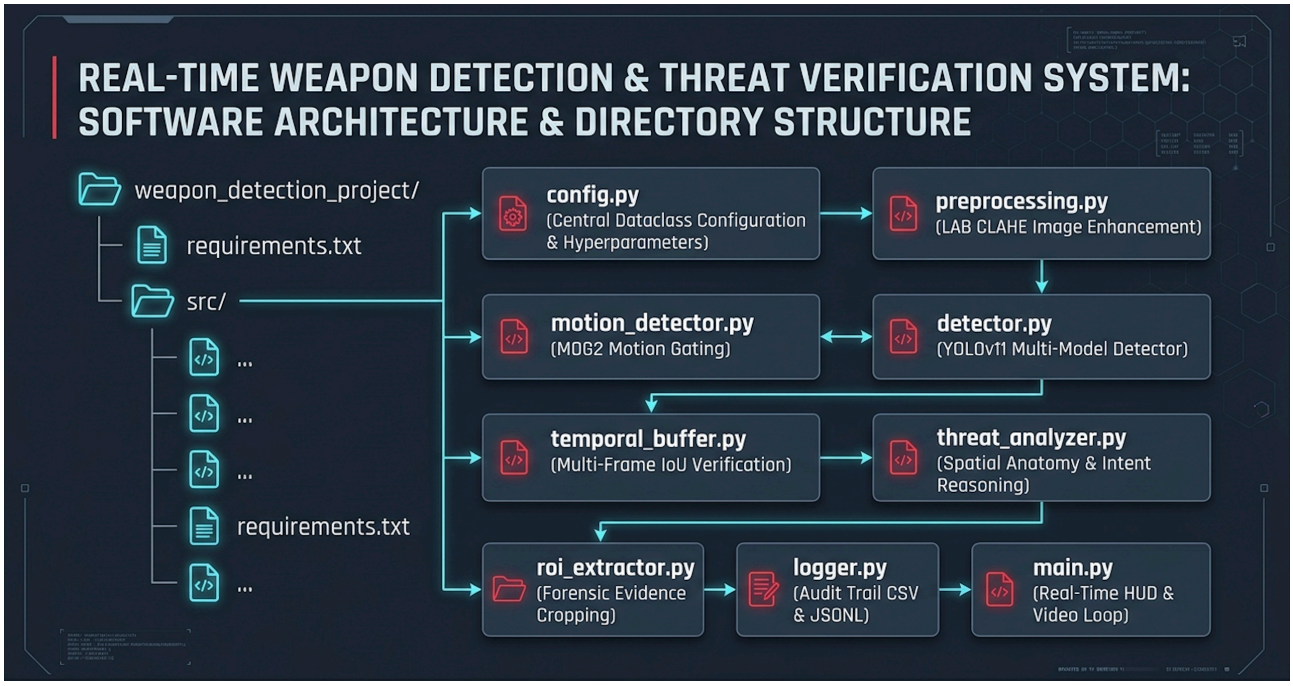


Fig 7.1 : Project Architecture & File Hierarchy

CHAPTER 8: SOURCE CODE

8.1 Overview of the Implementation Codebase

The Vision Guard project is developed using Python and integrates YOLOv11-based object detection with video processing, image enhancement, temporal verification, and evidence management. The codebase is organized into modules to simplify configuration, implementation, testing, and maintenance.

8.2 source code with file names

requirements.txt :

Dependencies for Computer Vision, Deep Learning, and Utilities:

```
# Core Computer Vision & Deep Learning
ultralytics>=8.3.0
opencv-python>=4.8.0
numpy>=1.24.0
torch>=2.0.0
torchvision>=0.15.0

# Data Management & Utilities
pandas>=2.0.0
python-dotenv>=1.0.0
pyyaml>=6.0.0
Pillow>=9.5.0
```

src/main.py :

Main Application Execution & Heads-Up Display (HUD):

```
"""
src/main.py
Main Application Entry Point for Real-Time Weapon Detection & Threat Verification System.
"""

import argparse
import sys
import time
from pathlib import Path
from typing import Union, List, Optional
import cv2
import numpy as np

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from src.config import AppConfig, config as default_config
```

```
from src.preprocessing import CLAHEPreprocessor
from src.motion_detector import MotionDetector
from src.detector import WeaponDetector, Detection
from src.temporal_buffer import TemporalVerificationBuffer, VerifiedThreat
from src.threat_analyzer import SpatialThreatAnalyzer, AnalyzedThreat
from src.roi_extractor import ROIExtractor
from src.logger import DetectionLogger


class WeaponDetectionPipeline:
    def __init__(self, app_config: AppConfig, source: Union[int, str]):
        self.config = app_config
        self.source = source
        self.preprocessor = CLAHEPreprocessor(self.config.clahe)
        self.motion_detector = MotionDetector(self.config.motion)
        self.detector = WeaponDetector(self.config.detector)
        self.threat_analyzer = SpatialThreatAnalyzer(self.config.threat)
        self.temporal_buffer = TemporalVerificationBuffer(self.config.temporal, self.config.threat)
        self.roi_extractor = ROIExtractor(self.config.output)
        self.logger = DetectionLogger(self.config.output)

    def run(self, headless: bool = False):
        source = int(self.source) if str(self.source).isdigit() else self.source
        cap = cv2.VideoCapture(source)

        if not cap.isOpened():
            print(f"[Error] Could not open video source: {source}")
            return

        window_name = "Real-Time Weapon Detection - YOLOv11 HUD"
        if not headless:
            cv2.namedWindow(window_name, cv2.WINDOW_NORMAL)

        try:
            while cap.isOpened():
                ret, frame = cap.read()
                if not ret:
                    break

                # 1. Motion & Preprocessing
                motion_detected, motion_area, _, _ = self.motion_detector.process(frame)
                enhanced_frame = self.preprocessor.apply(frame)

                # 2. Dual Context Inference
                raw_weapons, raw_persons, raw_context = [], [], []
                if motion_detected or not self.config.motion.enabled:
                    ctx = self.detector.detect_context(enhanced_frame)
                    raw_weapons, raw_persons, raw_context = ctx.weapons, ctx.persons, ctx.context_items
```

```
            # 3. Spatial Threat Analysis & Temporal Tracking
            analyzed = self.threat_analyzer.analyze(raw_weapons, raw_persons, raw_context)
            threats = self.temporal_buffer.update(raw_weapons)

            # Match verified threats with contextual reasoning
            for t in threats:
                for a in analyzed:
                    if self.temporal_buffer.compute_iou(a.weapon.box, t.detection.box) > 0.3:
                        t.threat_level = a.threat_level if t.is_verified else "CANDIDATE"
                        t.threat_score = a.threat_score
                        t.reason = a.reason
                        t.target_person_box = a.target_person_box
                        t.neck_zone_box = a.neck_zone_box

            # 4. Draw HUD & Annotations
            annotated = frame.copy()
            self._draw_hud(annotated, threats, raw_persons)

            # 5. Extract Evidence & Log
            for t in threats:
                if t.is_verified and t.threat_level in ("HIGH", "MID"):
                    self.roi_extractor.extract_and_save(frame, t.detection, t.track_id, annotated)
                    self.logger.log_event(t, source=str(self.source))

            if not headless:
                cv2.imshow(window_name, annotated)
                key = cv2.waitKey(1) & 0xFF
                if key in (ord('q'), 27):
                    break
                elif key in (ord('m'), ord('M')):
                    self.config.motion.enabled = not self.config.motion.enabled
                elif key in (ord('c'), ord('C')):
                    self.config.clahe.enabled = not self.config.clahe.enabled
        finally:
            cap.release()
            if not headless:
                cv2.destroyAllWindows()

    def _draw_hud(self, frame: np.ndarray, threats: List[VerifiedThreat], persons: List[Detection]):
        h, w = frame.shape[:2]

        # Draw Person Boxes
        for p in persons:
            cv2.rectangle(frame, (p.x1, p.y1), (p.x2, p.y2), (255, 200, 0), 1)
            cv2.putText(frame, "HUMAN", (p.x1 + 4, p.y1 + 14), cv2.FONT_HERSHEY_SIMPLEX,
0.4, (255, 200, 0), 1)
```

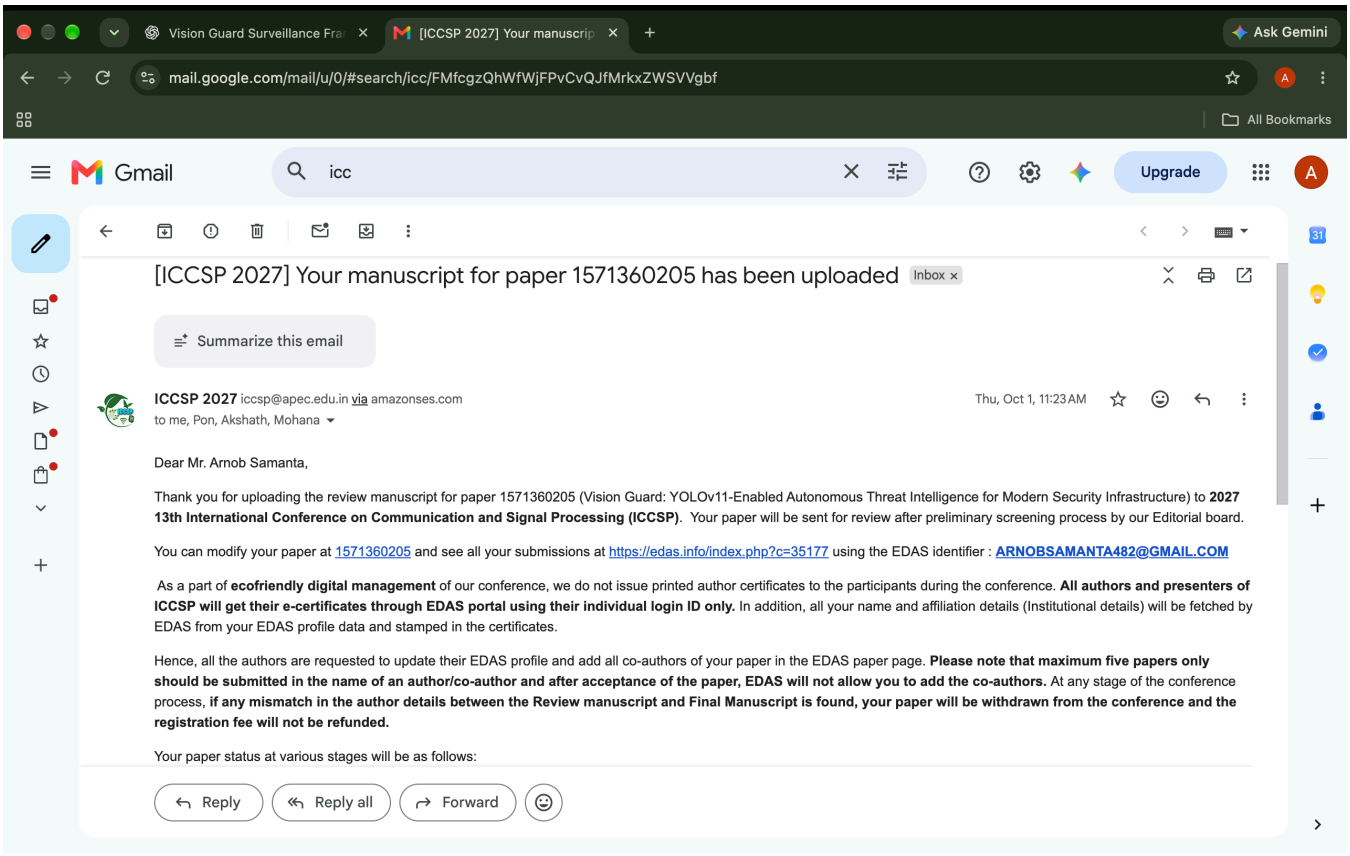
```
        # Draw Weapon Detections
        for t in threats:
            det = t.detection
            color = self.config.threat.color_high if t.threat_level == "HIGH" else (
                self.config.threat.color_mid if t.threat_level == "MID" else self.config.threat.color_low
            )
            cv2.rectangle(frame, (det.x1, det.y1), (det.x2, det.y2), color, 2)
            label = f"{t.threat_level} THREAT: {det.class_name.upper()} ({det.confidence:.2f})"
             cv2.putText(frame, label, (det.x1, max(det.y1 - 8, 15)), cv2.FONT_HERSHEY_SIMPLEX,
0.5, color, 2)


def main():
            parser = argparse.ArgumentParser(description="Real-Time Weapon Detection & Threat
Verification System")
    parser.add_argument("--source", type=str, default="0", help="Webcam index (0) or path to video
file.")
    parser.add_argument("--headless", action="store_true", help="Run without GUI window.")
    args = parser.parse_args()

    pipeline = WeaponDetectionPipeline(app_config=default_config, source=args.source)
    pipeline.run(headless=args.headless)


if __name__ == "__main__":
    main()
```

CHAPTER 9: CONFERENCE APPLICATION



Conference:2027 13th International Conference on Communication and Signal Processing (ICCSP)

Paper ID:1571360205